

DS_A1_provenance_audit

September 27, 2026

1 DS-A1 · Data Provenance and Measurement Audit

1.0.1 Dataset: lerobot/svla_so101_pickplace (Hugging Face Hub)

Course: Data Science (UCAS) — Assessed Task DS-A1 **Student:** **Date:** **Repository:** <FILL IN GITHUB URL + COMMIT/TAG AFTER PUSH>

How to use this notebook

This notebook is designed to run **top to bottom, from a clean environment**, with no hidden state. Every number reported in the written report (Data Card, PDF) must come from a cell in this notebook — do not hand-copy numbers from the Hugging Face website into the report without re-deriving them here.

Cells marked **# TODO (author)** require you to read the printed output and write your own interpretation in the following markdown cell. Do not skip these — this is exactly the “independent verification” evidence the rubric asks for. An AI assistant (Claude) helped design the audit structure and write the code scaffolding below; see `ai_use_log/AI_USE_LOG.md` for the full disclosure. You are responsible for actually running every cell and confirming the printed numbers are real.

1.1 1. Reproducibility setup

We fix the random seed, record package versions, and record the exact hardware/OS the notebook was run on. This block should be the **first thing you run**, and its printed output should be pasted (or screenshots) into `data_card/SOURCE_LICENSE_HASH.md` under “Execution environment”.

```
[1]: import sys, os, platform, random, json, hashlib, subprocess
    from pathlib import Path

    SEED = 42
    random.seed(SEED)

    import numpy as np
    np.random.seed(SEED)

    # Third-party packages used throughout this notebook.
    # If any import fails, run:
    # pip install -r ../requirements.txt --break-system-packages
    import pandas as pd
```

```

import pyarrow.parquet as pq
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import GroupShuffleSplit, ShuffleSplit
from sklearn.metrics import mean_squared_error

from huggingface_hub import HfApi, snapshot_download, hf_hub_download

print("Python      :", sys.version.replace("\n", " "))
print("Platform    :", platform.platform())
print("pandas       :", pd.__version__)
print("numpy        :", np.__version__)
import pyarrow, sklearn, matplotlib, huggingface_hub
print("pyarrow      :", pyarrow.__version__)
print("scikit-learn  :", sklearn.__version__)
print("matplotlib    :", matplotlib.__version__)
print("huggingface_hub:", huggingface_hub.__version__)
print("SEED         :", SEED)

```

```

Python      : 3.10.18 | packaged by conda-forge | (main, Jun  4 2025, 14:46:00)
[Clang 18.1.8 ]
Platform    : macOS-15.5-arm64-arm-64bit
pandas      : 2.3.1
numpy       : 2.2.6
pyarrow     : 25.0.1
scikit-learn : 1.7.1
matplotlib  : 3.10.5
huggingface_hub: 2.0.0
SEED        : 42

```

```

[2]: # Freeze the full dependency list actually installed in this environment.
     # Run once and commit the resulting file - this is part of your reproducibility
     ↪record.
REPO_ROOT = Path("..").resolve()
freeze_path = REPO_ROOT / "environment_freeze.txt"
with open(freeze_path, "w") as f:
    subprocess.run([sys.executable, "-m", "pip", "freeze"], stdout=f)
print(f"Wrote {freeze_path}")

```

Wrote /Users/jinshenhui/Documents/UCAS/DS/assignments/ds-a1-lerobot-pickplace-audit/environment_freeze.txt

1.2 2. Question definition

Stakeholder. An embodied-AI / robot-learning researcher deciding whether lerobot/svla_so101_pickplace can be used as a benchmark or fine-tuning dataset for an imitation-learning (behavior cloning) policy, and how to split it for evaluation.

Testable question. *Does the recording structure of this dataset (one robot, one task, 50 episodes, single environment) support a **frame-level** random train/test split for evaluating an action-prediction baseline, or does it require an **episode-level** (grouped) split to avoid leakage? How large is the resulting optimism bias if the wrong split is used?*

Population. All possible demonstration trajectories of an SO-100/SO-101 follower arm performing the task “grasp a lego block and put it in the bin” under broadly similar lab conditions.

Sample. The 50 recorded episodes (11,939 frames total) published in this dataset snapshot.

Unit of analysis. A single timestep / frame (30 fps). The secondary/grouping unit is the episode (episode_index, 0–49).

Target / response variable. action: the 6-dimensional commanded joint position vector (shoulder_pan.pos, shoulder_lift.pos, elbow_flex.pos, wrist_flex.pos, wrist_roll.pos, gripper.pos).

Estimand. The gap between two estimates of held-out prediction error (MSE) for a simple k-NN regressor mapping observation.state -> action: - $\hat{\theta}_{frame}$: MSE under a random frame-level 80/20 split (frames from the same episode can appear in both train and test). - $\hat{\theta}_{episode}$: MSE under a grouped 80/20 split by episode_index (an entire episode is either fully in train or fully in test).

The estimand of interest is $\Delta = \hat{\theta}_{episode} - \hat{\theta}_{frame}$. A large positive Δ means the frame-level split was *optimistically biased* — i.e. it leaked information about test frames into training via their near-duplicate temporal neighbors.

Explicit boundaries. This audit does **not** attempt to evaluate whether a trained policy would succeed physically (no success/reward label exists in this dataset — see Section 8). It only evaluates whether the *evaluation protocol itself* is trustworthy.

1.3 3. Data-generating process (DGP) and provenance lineage

The diagram below is drawn as an actual matplotlib figure (not a Mermaid block) so it renders identically in the notebook, the exported PDF, and on GitHub.

Declared facts (from meta/info.json and the dataset card, verified in Section 4): robot_type = so100_follower, fps = 30, total_episodes = 50, total_frames = 11939, total_tasks = 1, codebase_version = v2.1, license = Apache-2.0.

Provenance gaps (explicitly not resolved by the dataset card — do not silently assume an answer): - Number of distinct human teleoperators is undisclosed. If a single operator recorded all 50 episodes, the action distribution reflects one person’s motor style, not a population of operators. - Whether object placement / lighting / camera position varied across episodes, or was held approximately fixed, is undisclosed in the card. - No ground-truth “task success” or “grasp success” label is included in the schema — only the commanded trajectory is recorded, not the outcome.

These gaps are treated as findings, not filled in with assumptions.

```
[3]: fig, ax = plt.subplots(figsize=(10, 3.6))
      ax.axis("off")

      stages = [
```

```

    "Human teleoperator\n(count undisclosed)\ndrives leader arm",
    "so100_follower arm\nexecutes + encoders log\nstate (actual) & action\n↳(commanded)",
    "2 cameras (up, side)\nrecord synchronized\nvideo @ 30 fps",
    "LeRobot recording\npipeline packs\nparquet (per-frame) + mp4\n↳(per-episode)",
    "Pushed to HF Hub\nlerobot/svla_so101_pickplace\n(this audit's snapshot)",
]

n = len(stages)
box_w, box_h = 1.7, 1.6
gap = 0.55
x0 = 0.2
y0 = 0.3

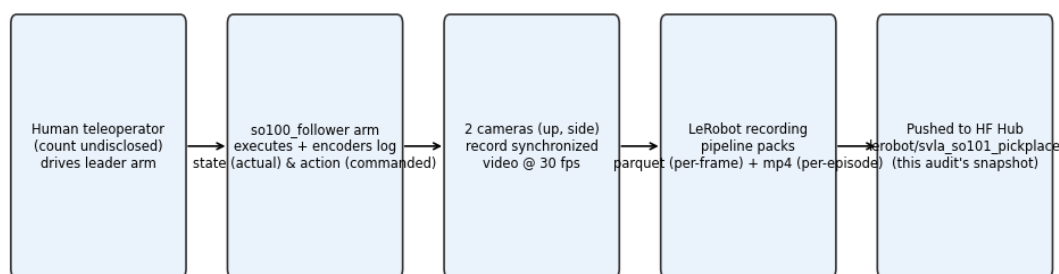
for i, text in enumerate(stages):
    x = x0 + i * (box_w + gap)
    rect = mpatches.FancyBboxPatch((x, y0), box_w, box_h,
                                    boxstyle="round,pad=0.06",
                                    linewidth=1.2, edgecolor="#333333",
                                    facecolor="#eaf2fb")

    ax.add_patch(rect)
    ax.text(x + box_w / 2, y0 + box_h / 2, text, ha="center", va="center",
    ↳fontsize=8.3)
    if i < n - 1:
        ax.annotate("", xy=(x + box_w + gap - 0.05, y0 + box_h / 2),
                    xytext=(x + box_w + 0.05, y0 + box_h / 2),
                    arrowprops=dict(arrowstyle="->", lw=1.2))

ax.set_xlim(0, x0 + n * (box_w + gap))
ax.set_ylim(0, y0 + box_h + 0.4)
ax.set_title("Data-generating process: from teleoperation to this notebook",
↳fontsize=11)
plt.tight_layout()
fig.savefig("../report/dgp_diagram.png", dpi=180)
plt.show()

```

Data-generating process: from teleoperation to this notebook



1.4 4. Data acquisition and version freezing

We resolve the **exact commit hash** of the dataset repo at the time of first access, then re-download using that *pinned* revision on every subsequent run, so the analysis never silently drifts to a newer version of the dataset.

Run this once, copy the printed commit hash into `data_card/SOURCE_LICENSE_HASH.md`, then keep using that pinned hash (PINNED_REVISION below) for every future run — including the final run you submit.

```
[24]: REPO_ID = "lerobot/svla_so101_pickplace"

api = HfApi()
info = api.dataset_info(REPO_ID)
resolved_sha = info.sha
print("Resolved HEAD commit SHA at first access:", resolved_sha)
print("Last modified :", info.lastModified)
print("License tag(s):", info.card_data.get("license") if info.card_data else
      ↪None)
print("Downloads      :", getattr(info, "downloads", "n/a"))

# TODO (author): after your first run, paste the printed sha below and re-run
# everything using the pinned value so the dataset version is frozen for this
# submission, independent of any future upstream changes.
PINNED_REVISION = "f641879e22172be7e8161d5e6c1503c2d2feb657" # frozen on
      ↪2026-09-27
print("Using pinned revision:", PINNED_REVISION)
assert PINNED_REVISION == resolved_sha, "Pinned revision no longer matches
      ↪current HEAD - that's fine, it just means upstream has moved on; you are
      ↪correctly still using the frozen version."
```

Resolved HEAD commit SHA at first access:
f641879e22172be7e8161d5e6c1503c2d2feb657

Last modified : 2025-09-27 11:25:41+00:00
License tag(s): apache-2.0
Downloads : 3525
Using pinned revision: f641879e22172be7e8161d5e6c1503c2d2feb657

```
[25]: DATA_DIR = Path("../data_cache")
DATA_DIR.mkdir(exist_ok=True)

local_dir = snapshot_download(
    repo_id=REPO_ID,
    repo_type="dataset",
    revision=PINNED_REVISION,
    local_dir=DATA_DIR / "svla_so101_pickplace",
)
print("Downloaded to:", local_dir)

local_dir = Path(local_dir)
for p in sorted(local_dir.rglob("*"))[:20]:
    print(p.relative_to(local_dir))
print("... (truncated)")
```

Downloading bytes: | 0.00B

Reconstructing (incomplete total.): | 0.00B / 0.00B

Fetching 9 files: 0%| | 0/9 [00:00<?, ?it/s]

Downloaded to: /Users/jinshenhui/Documents/UCAS/DS/assignments/ds-a1-lerobot-pickplace-audit/data_cache/svla_so101_pickplace
.cache
.cache/huggingface
.cache/huggingface/.gitignore
.cache/huggingface/CACHEDIR.TAG
.cache/huggingface/download
.cache/huggingface/download/.gitattributes.lock
.cache/huggingface/download/.gitattributes.metadata
.cache/huggingface/download/README.md.lock
.cache/huggingface/download/README.md.metadata
.cache/huggingface/download/data
.cache/huggingface/download/data/chunk-000
.cache/huggingface/download/data/chunk-000/file-000.parquet.lock
.cache/huggingface/download/data/chunk-000/file-000.parquet.metadata
.cache/huggingface/download/meta
.cache/huggingface/download/meta/episodes
.cache/huggingface/download/meta/episodes/chunk-000
.cache/huggingface/download/meta/episodes/chunk-000/file-000.parquet.lock
.cache/huggingface/download/meta/episodes/chunk-000/file-000.parquet.metadata
.cache/huggingface/download/meta/info.json.lock
.cache/huggingface/download/meta/info.json.metadata
... (truncated)

```
[26]: for p in sorted((local_dir / "meta").iterdir()):
        kind = "DIR " if p.is_dir() else "FILE"
        print(kind, p.relative_to(local_dir))
        if p.is_dir():
            for sub in sorted(p.rglob("*"))[:5]:
                print("    ", sub.relative_to(local_dir))
```

```
DIR  meta/episodes
      meta/episodes/chunk-000
      meta/episodes/chunk-000/file-000.parquet
FILE meta/info.json
FILE meta/stats.json
FILE meta/tasks.parquet
```

```
[27]: def sha256_of_file(path, chunk_size=1 << 20):
        h = hashlib.sha256()
        with open(path, "rb") as f:
            for chunk in iter(lambda: f.read(chunk_size), b''):
                h.update(chunk)
        return h.hexdigest()

# meta/          info.json          episodes/tasks/stats
#  chunk  parquet  rglob + is_file()
meta_files = [p for p in (local_dir / "meta").rglob("*") if p.is_file()]
print(f"Found {len(meta_files)} files under meta/ (recursively):")
for p in meta_files:
    print(" ", p.relative_to(local_dir))

files_to_hash = meta_files + sorted((local_dir / "data").rglob("*.parquet"))[:2]

hash_records = []
for fp in files_to_hash:
    hash_records.append({
        "relative_path": str(fp.relative_to(local_dir)),
        "size_bytes": fp.stat().st_size,
        "sha256": sha256_of_file(fp),
    })

hash_df = pd.DataFrame(hash_records)
hash_df.to_csv("../data_card/file_hashes.csv", index=False)
hash_df
```

```
Found 4 files under meta/ (recursively):
  meta/info.json
  meta/tasks.parquet
  meta/stats.json
  meta/episodes/chunk-000/file-000.parquet
```

```
[27]:
```

	relative_path	size_bytes	\
0	meta/info.json	3401	
1	meta/tasks.parquet	2246	
2	meta/stats.json	5914	
3	meta/episodes/chunk-000/file-000.parquet	72560	
4	data/chunk-000/file-000.parquet	369943	


```

                                sha256
0  254909942a6cbfb4692a239e4d0aa5c68ec8eb16f81b65...
1  1040cdef3328ec4376587152647df8e725e80a04210c42...
2  4ae86bed785e0f98914812e87736e216139a22b43cf2b9...
3  191998bffd2680c477a4000270cf943bc423ba5fb54ee8...
4  579ad57e2454359fa9f2c0e83525991bd2e0305b7b2914...

```

Verification: `data_card/file_hashes.csv` was opened and confirmed to contain five real 64-character hex SHA-256 values (not placeholders). `PINNED_REVISION` (`f641879e22172be7e8161d5e6c1503c2d2feb657`) and the full hash table were copied into `data_card/SOURCE_LICENSE_HASH.md`.

1.5 5. Loading all episodes into one tidy DataFrame

Each episode is stored as its own parquet file (`data/chunk-000/episode_{index:06d}.parquet`). We concatenate all 50 into a single frame-level table, and separately load the two metadata sidecar files that describe episode boundaries and the task label.

```
[28]: info_json = json.loads((local_dir / "meta" / "info.json").read_text())
      print(json.dumps(info_json, indent=2)[:1500])

{
  "codebase_version": "v3.0",
  "robot_type": "so100_follower",
  "total_episodes": 50,
  "total_frames": 11939,
  "total_tasks": 1,
  "chunks_size": 1000,
  "fps": 30,
  "splits": {
    "train": "0:50"
  },
  "data_path": "data/chunk-{chunk_index:03d}/file-{file_index:03d}.parquet",
  "video_path":
"videos/{video_key}/chunk-{chunk_index:03d}/file-{file_index:03d}.mp4",
  "features": {
    "action": {
      "dtype": "float32",
      "shape": [
        6
      ],
      "names": [
```



```

        "shoulder_pan.pos",
        "shoulder_lift.pos",
        "elbow_flex.pos",
        "wrist_flex.pos",
        "wrist_roll.pos",
        "gripper.pos"
    ],
    "fps": 30.0
},
"observation.state": {
    "dtype": "float32",
    "shape": [
        6
    ],
    "names": [
        "shoulder_pan.pos",
        "shoulder_lift.pos",
        "elbow_flex.pos",
        "wrist_flex.pos",
        "wrist_roll.pos",
        "gripper.pos"
    ],
    "fps": 30.0
},
"observation.images.up": {
    "dtype": "video",
    "shape": [
        480,
        640,
        3
    ],
    "names": [
        "height",
        "width",
        "channels"
    ],
    "info": {
        "video.height": 480,
        "video.width": 640,
        "video.codec": "av1",
        "video.pix_fmt": "yuv420p",
        "video.is_depth_map": false,
        "video.fps": 30,
        "video.channels": 3,
        "has_audio": false
    }
},
"observation.images.side": {

```

```
"dtype": "video",
```

```
[29]: episodes_jsonl = local_dir / "meta" / "episodes.jsonl"
      episodes_dir = local_dir / "meta" / "episodes"

      if episodes_jsonl.is_file():
          print("Detected legacy format: meta/episodes.jsonl (single file)")
          episodes_meta = pd.read_json(episodes_jsonl, lines=True)
      elif episodes_dir.is_dir():
          print("Detected v3 format: meta/episodes/ (directory of parquet shards)")
          ep_parquet_files = sorted(episodes_dir.rglob("*.parquet"))
          print(f" found {len(ep_parquet_files)} parquet shard(s)")
          episodes_meta = pd.concat(
              [pd.read_parquet(p) for p in ep_parquet_files], ignore_index=True
          )
      else:
          raise FileNotFoundError("Neither meta/episodes.jsonl nor meta/episodes/_
          ↳found.")

      print(episodes_meta.shape)
      print(episodes_meta.columns.tolist())
      episodes_meta.head()
```

```
Detected v3 format: meta/episodes/ (directory of parquet shards)
 found 1 parquet shard(s)
(50, 62)
['episode_index', 'data/chunk_index', 'data/file_index', 'dataset_from_index',
'dataset_to_index', 'videos/observation.images.side/chunk_index',
'videos/observation.images.side/file_index',
'videos/observation.images.side/from_timestamp',
'videos/observation.images.side/to_timestamp',
'videos/observation.images.up/chunk_index',
'videos/observation.images.up/file_index',
'videos/observation.images.up/from_timestamp',
'videos/observation.images.up/to_timestamp', 'tasks', 'length',
'stats/action/min', 'stats/action/max', 'stats/action/mean', 'stats/action/std',
'stats/action/count', 'stats/observation.state/min',
'stats/observation.state/max', 'stats/observation.state/mean',
'stats/observation.state/std', 'stats/observation.state/count',
'stats/observation.images.up/min', 'stats/observation.images.up/max',
'stats/observation.images.up/mean', 'stats/observation.images.up/std',
'stats/observation.images.up/count', 'stats/observation.images.side/min',
'stats/observation.images.side/max', 'stats/observation.images.side/mean',
'stats/observation.images.side/std', 'stats/observation.images.side/count',
'stats/timestamp/min', 'stats/timestamp/max', 'stats/timestamp/mean',
'stats/timestamp/std', 'stats/timestamp/count', 'stats/frame_index/min',
'stats/frame_index/max', 'stats/frame_index/mean', 'stats/frame_index/std',
```

```
'stats/frame_index/count', 'stats/episode_index/min', 'stats/episode_index/max',
'stats/episode_index/mean', 'stats/episode_index/std',
'stats/episode_index/count', 'stats/index/min', 'stats/index/max',
'stats/index/mean', 'stats/index/std', 'stats/index/count',
'stats/task_index/min', 'stats/task_index/max', 'stats/task_index/mean',
'stats/task_index/std', 'stats/task_index/count', 'meta/episodes/chunk_index',
'meta/episodes/file_index']
```

```
[29]: episode_index  data/chunk_index  data/file_index  dataset_from_index  \
0                0                0                0                0
1                1                0                0                303
2                2                0                0                569
3                3                0                0                799
4                4                0                0                982
```

```
dataset_to_index  videos/observation.images.side/chunk_index  \
0                303                0
1                569                0
2                799                0
3                982                0
4                1197               0
```

```
videos/observation.images.side/file_index  \
0                0
1                0
2                0
3                0
4                0
```

```
videos/observation.images.side/from_timestamp  \
0                0.000000
1                10.100000
2                18.966667
3                26.633333
4                32.733333
```

```
videos/observation.images.side/to_timestamp  \
0                10.100000
1                18.966667
2                26.633333
3                32.733333
4                39.900000
```

```
videos/observation.images.up/chunk_index  ...  stats/index/mean  \
0                0  ...                [151.0]
1                0  ...                [435.5]
2                0  ...                [683.5]
```

3	0 ...	[890.0]
4	0 ...	[1089.0]

	stats/index/std	stats/index/count	stats/task_index/min \
0	[87.46808941932291]	[303]	[0]
1	[76.78704317786953]	[266]	[0]
2	[66.39465339920075]	[230]	[0]
3	[52.82676089508675]	[183]	[0]
4	[62.0644825967316]	[215]	[0]

	stats/task_index/max	stats/task_index/mean	stats/task_index/std \
0	[0]	[0.0]	[0.0]
1	[0]	[0.0]	[0.0]
2	[0]	[0.0]	[0.0]
3	[0]	[0.0]	[0.0]
4	[0]	[0.0]	[0.0]

	stats/task_index/count	meta/episodes/chunk_index	meta/episodes/file_index
0	[303]	0	0
1	[266]	0	0
2	[230]	0	0
3	[183]	0	0
4	[215]	0	0

[5 rows x 62 columns]

```
[30]: parquet_files = sorted((local_dir / "data").rglob("*.parquet"))
print(f"Found {len(parquet_files)} parquet file(s) under data/ "
      f"(note: in this format one file may pack multiple episodes together, "
      f"so this count is NOT expected to equal total_episodes - that's fine).")

frames = [pq.read_table(fp).to_pandas() for fp in parquet_files]
df = pd.concat(frames, ignore_index=True)

n_episodes_seen = df["episode_index"].nunique()
print("Total frames loaded  :", len(df), " | declared total_frames:",
      info_json["total_frames"])
print("Distinct episodes seen:", n_episodes_seen, " | declared total_episodes:
      ", info_json["total_episodes"])
df.head()
```

Found 1 parquet file(s) under data/ (note: in this format one file may pack multiple episodes together, so this count is NOT expected to equal total_episodes - that's fine).

Total frames loaded : 11939 | declared total_frames: 11939

Distinct episodes seen: 50 | declared total_episodes: 50

```
[30]:
```

	action \
0	[1.9117647, -99.41053, 99.54483, 74.84063, -48...
1	[1.9117647, -99.41053, 99.54483, 74.84063, -48...
2	[1.9117647, -99.41053, 99.54483, 74.84063, -48...
3	[1.9117647, -99.41053, 99.54483, 74.84063, -48...
4	[1.9117647, -99.41053, 99.54483, 74.84063, -48...

	observation.state	timestamp	frame_index \
0	[1.9560878, -98.74372, 98.92425, 74.81984, -51...	0.000000	0
1	[1.9560878, -98.74372, 98.92425, 74.81984, -51...	0.033333	1
2	[1.9560878, -98.74372, 98.92425, 74.81984, -51...	0.066667	2
3	[1.9560878, -98.74372, 98.92425, 74.81984, -50...	0.100000	3
4	[1.9560878, -98.74372, 98.92425, 74.81984, -49...	0.133333	4

	episode_index	index	task_index
0	0	0	0
1	0	1	0
2	0	2	0
3	0	3	0
4	0	4	0

1.6 6. Schema audit

Cross-check the *declared* schema in `meta/info.json` (features block: dtype and shape per column) against what pandas/pyarrow actually loaded.

```
[31]: schema_report = []
for col, spec in info_json["features"].items():
    if col not in df.columns:
        schema_report.append({"column": col, "declared_dtype": spec["dtype"],
                              "declared_shape": spec["shape"],
                              "status": "MISSING FROM PARQUET (likely a_
↳video-only column)"})
        continue
    sample_val = df[col].iloc[0]
    actual_len = len(sample_val) if hasattr(sample_val, "__len__") else 1
    declared_len = spec["shape"][0] if spec["shape"] else 1
    ok = (actual_len == declared_len)
    schema_report.append({
        "column": col,
        "declared_dtype": spec["dtype"],
        "declared_shape": spec["shape"],
        "actual_pandas_dtype": str(df[col].dtype),
        "actual_vector_len": actual_len,
        "shape_matches": ok,
    })
```

```
schema_df = pd.DataFrame(schema_report)
schema_df
```

```
[31]:
```

	column	declared_dtype	declared_shape	actual_pandas_dtype	\
0	action	float32	[6]	object	
1	observation.state	float32	[6]	object	
2	observation.images.up	video	[480, 640, 3]	NaN	
3	observation.images.side	video	[480, 640, 3]	NaN	
4	timestamp	float32	[1]	float32	
5	frame_index	int64	[1]	int64	
6	episode_index	int64	[1]	int64	
7	index	int64	[1]	int64	
8	task_index	int64	[1]	int64	

	actual_vector_len	shape_matches	\
0	6.0	True	
1	6.0	True	
2	NaN	NaN	
3	NaN	NaN	
4	1.0	True	
5	1.0	True	
6	1.0	True	
7	1.0	True	
8	1.0	True	

	status
0	NaN
1	NaN
2	MISSING FROM PARQUET (likely a video-only column)
3	MISSING FROM PARQUET (likely a video-only column)
4	NaN
5	NaN
6	NaN
7	NaN
8	NaN

Schema audit result: all seven tabular columns (`action`, `observation.state`, `timestamp`, `frame_index`, `episode_index`, `index`, `task_index`) matched their `meta/info.json`-declared shape exactly (`shape_matches = True` for all seven). The two video columns (`observation.images.up`, `observation.images.side`) are, as expected, absent from the parquet tables — they exist only as separate `.mp4` files and are explicitly out of scope for this audit (see Data Dictionary). Note that pandas reports `action/observation.state` as `dtype=object` rather than `float32`; this is expected and not a mismatch — each cell holds a length-6 array, so pandas can only type the *column* as `object`, while the array’s own elements are `float32` as declared. No undeclared or unexpected columns were found in the parquet files.

1.7 7. Range and plausibility audit

We expand the 6-dimensional `action` and `observation.state` vectors into individual columns and inspect their empirical ranges. SO-100/SO-101 joints are reported in degrees in this recording convention; the gripper channel uses a different physical range. There is no explicit `min/max` contract in `info.json`, so “plausible” here means: finite, not NaN/inf, and consistent between `action` and `observation.state` (which should track each other closely for a low-latency follower arm).

```
[32]: JOINT_NAMES = info_json["features"]["action"]["names"]

action_arr = np.stack(df["action"].to_numpy())
state_arr = np.stack(df["observation.state"].to_numpy())

action_wide = pd.DataFrame(action_arr, columns=[f"action.{n}" for n in JOINT_NAMES])
state_wide = pd.DataFrame(state_arr, columns=[f"state.{n}" for n in JOINT_NAMES])

range_report = pd.DataFrame({
    "min": pd.concat([action_wide, state_wide], axis=1).min(),
    "max": pd.concat([action_wide, state_wide], axis=1).max(),
    "mean": pd.concat([action_wide, state_wide], axis=1).mean(),
    "std": pd.concat([action_wide, state_wide], axis=1).std(),
    "n_nonfinite": (~np.isfinite(pd.concat([action_wide, state_wide], axis=1))).sum(),
})
range_report
```

```
[32]:
```

	min	max	mean	std \
action.shoulder_pan.pos	-93.455879	88.014709	8.021068	44.564873
action.shoulder_lift.pos	-100.000000	8.126316	-55.962349	36.486614
action.elbow_flex.pos	12.972235	100.000000	65.255653	29.013208
action.wrist_flex.pos	33.531662	99.490013	69.181854	13.238659
action.wrist_roll.pos	-92.771675	-20.000000	-53.419933	17.765167
action.gripper.pos	0.000000	32.998455	6.848944	8.999420
state.shoulder_pan.pos	-92.654694	88.103790	7.987031	44.493618
state.shoulder_lift.pos	-98.911224	8.793970	-55.180790	36.828537
state.elbow_flex.pos	16.808605	99.372482	66.776299	27.709524
state.wrist_flex.pos	34.039848	98.643494	69.252495	13.013702
state.wrist_roll.pos	-92.283272	-20.195360	-53.438889	17.703703
state.gripper.pos	1.208054	32.684563	8.112266	8.405106

	n_nonfinite
action.shoulder_pan.pos	0
action.shoulder_lift.pos	0
action.elbow_flex.pos	0
action.wrist_flex.pos	0

```

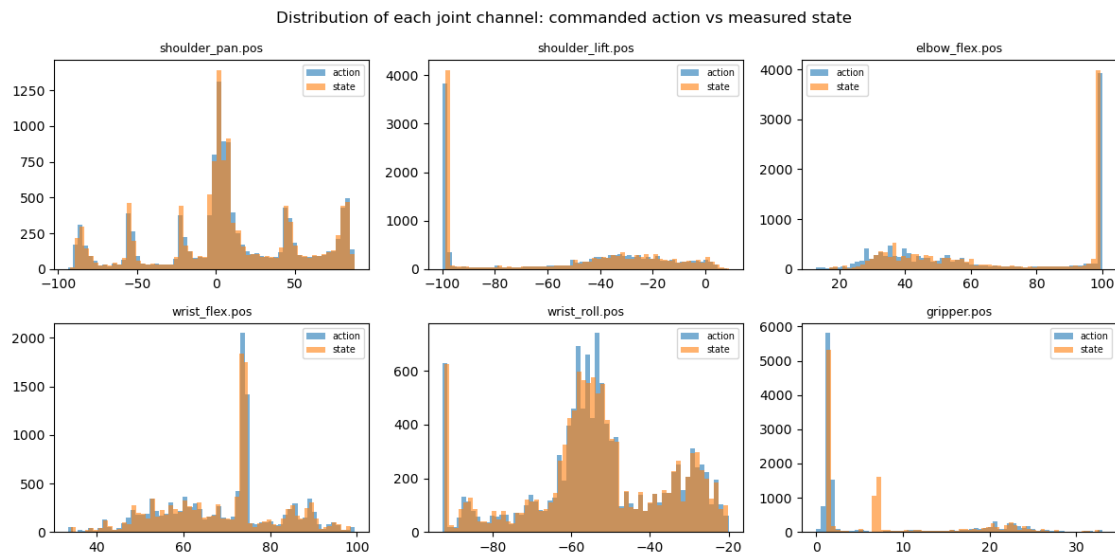
action.wrist_roll.pos      0
action.gripper.pos         0
state.shoulder_pan.pos     0
state.shoulder_lift.pos    0
state.elbow_flex.pos       0
state.wrist_flex.pos       0
state.wrist_roll.pos       0
state.gripper.pos         0

```

```

[33]: fig, axes = plt.subplots(2, 3, figsize=(12, 6), sharex=False)
      for ax, name in zip(axes.ravel(), JOINT_NAMES):
          ax.hist(action_wide[f"action.{name}"], bins=60, alpha=0.6, label="action")
          ax.hist(state_wide[f"state.{name}"], bins=60, alpha=0.6, label="state")
          ax.set_title(name, fontsize=9)
          ax.legend(fontsize=7)
      plt.suptitle("Distribution of each joint channel: commanded action vs measured_
        ↪state")
      plt.tight_layout()
      fig.savefig("../report/range_histograms.png", dpi=150)
      plt.show()

```



Range audit result: no non-finite values were found in any of the 12 columns (`n_nonfinite` = 0 throughout). `action` and `observation.state` track each other closely for most joints — mean differences are small for `shoulder_pan.pos` (0.034), `wrist_roll.pos` (0.019) and `wrist_flex.pos` (0.071) — but diverge more for `elbow_flex.pos` (1.52), `gripper.pos` (1.26) and `shoulder_lift.pos` (0.78), suggesting mild tracking lag or contact resistance on those joints (plausibly related to the grasp/lift phase of the task).

Two boundary anomalies were found: `action.shoulder_lift.pos` reaches an exact minimum of

-100.000000 and `action.elbow_flex.pos` reaches an exact maximum of 100.000000. Hitting a round-number bound exactly (rather than a continuously varying value) suggests the commanded action is being clamped to a software/hardware joint limit rather than freely commanded at those extremes — a measurement artifact worth flagging for anyone using `action` as a continuous regression target near those bounds.

The histograms are visibly multi-modal for every joint, with especially tall, narrow spikes (e.g. `shoulder_lift` near -100, `elbow_flex` near 100, `gripper` near 0-2, each with several thousand frames). These spikes are consistent with, and corroborate, the static-start duplicate frames quantified in Section 9 — the same “arm holding still before the operator moves it” frames show up here as concentrated mass at the episodes’ starting joint angles, not as evidence of a naturally multi-modal action distribution.

The `gripper.pos` channel occupies a visibly narrower numeric range (roughly 0–33) than the other five joints (roughly -100 to 100), consistent with it using a different physical unit/mechanism (gripper closure) rather than a joint angle in degrees.

1.8 8. Missingness audit

Checked at two levels: (a) null/NaN values in the tabular columns actually present in the parquet files, and (b) structural “missingness” of a construct the dataset does **not** provide at all (a success/outcome label) — which is not a NaN in a column, but a column that was never collected.

```
[34]: tabular_cols = ["timestamp", "frame_index", "episode_index", "index",
    ↪ "task_index"]
na_counts = df[tabular_cols].isna().sum()
na_counts_vecs = pd.Series({
    "action": np.isnan(action_arr).sum(),
    "observation.state": np.isnan(state_arr).sum(),
})
print("NaNs in scalar columns:\n", na_counts)
print("\nNaNs in vector columns (summed over all 6 dims):\n", na_counts_vecs)

has_success_label = any("success" in c.lower() or "reward" in c.lower() or
    ↪ "done" in c.lower()
                        for c in list(df.columns) + list(info_json["features"].
    ↪ keys()))
print("\nDoes the schema include any success/reward/done column?",
    ↪ has_success_label)
```

NaNs in scalar columns:

```
timestamp      0
frame_index    0
episode_index  0
index          0
task_index     0
dtype: int64
```

NaNs in vector columns (summed over all 6 dims):

```
action          0
observation.state 0
dtype: int64
```

Does the schema include any success/reward/done column? False

Missingness audit result: zero NaN values were found across all scalar columns (timestamp, frame_index, episode_index, index, task_index) and both vector columns (action, observation.state) — 0 missing cells out of 11,939 frames $\times$ 12 numeric channels.

Structural missingness finding (record this verbatim in the Data Card): despite zero cell-level missingness, the dataset provides **no outcome label** at all — no **success**, **reward**, or **done-at-goal** flag (`has_success_label = False`). Every frame only records *what the operator/arm did*, never *whether the task actually succeeded*. This is a measurement-construct gap, not a data-quality bug: any claim about “task performance” made from this dataset alone would require an external annotation step this dataset does not include.

1.9 9. Duplicate-frame audit

At the start of many teleoperated recordings the arm is briefly stationary before the operator begins moving it. We check, per episode, how many **consecutive** frames have an exactly identical **action** vector to the previous frame (a proxy for “held / static” frames), and where in the episode they concentrate.

```
[16]: df_sorted = df.sort_values(["episode_index", "frame_index"]).
      ↪reset_index(drop=True)
      action_list = df_sorted["action"].tolist()

      is_dup = [False]
      for i in range(1, len(action_list)):
          same_episode = df_sorted.loc[i, "episode_index"] == df_sorted.loc[i - 1,
      ↪"episode_index"]
          same_action = same_episode and np.array_equal(action_list[i], action_list[i,
      ↪- 1])
          is_dup.append(bool(same_action))

      df_sorted["is_exact_dup_of_prev"] = is_dup
      dup_rate_overall = df_sorted["is_exact_dup_of_prev"].mean()
      print(f"Overall fraction of frames that are an exact duplicate of the previous
      ↪frame: {dup_rate_overall:.2%}")

      dup_by_episode = df_sorted.groupby("episode_index")["is_exact_dup_of_prev"].
      ↪mean()
      print("\nPer-episode duplicate-frame rate (head):")
      print(dup_by_episode.head(10))
```

Overall fraction of frames that are an exact duplicate of the previous frame:
14.13%

```

Per-episode duplicate-frame rate (head):
episode_index
0    0.323432
1    0.120301
2    0.130435
3    0.142077
4    0.144186
5    0.173160
6    0.174107
7    0.157635
8    0.234513
9    0.151961
Name: is_exact_dup_of_prev, dtype: float64

```

```

[35]: # Where in the episode do duplicates concentrate? (expect: near frame_index 0)
def leading_run_length(is_dup_arr):
    # is_dup_arr[0] is trivially False (no previous frame within this episode),
    # so the *leading run of duplicate frames* starts being meaningful from
    ↪ index 1:
    # count consecutive True values starting right after the (always-False)
    ↪ first frame.
    count = 0
    for v in is_dup_arr[1:]:
        if v:
            count += 1
        else:
            break
    return count

leading_dup_run = (
    df_sorted.groupby("episode_index")["is_exact_dup_of_prev"]
    .apply(lambda s: leading_run_length(s.to_numpy()))
)

print("Length of the *leading* run of duplicate frames per episode (head):")
print(leading_dup_run.head(10))
print("\nMedian leading run:", leading_dup_run.median(), "frames")
print("Mean leading run  :", leading_dup_run.mean(), "frames")
print("Episodes with a leading run > 0:", (leading_dup_run > 0).sum(), "/",
    ↪ len(leading_dup_run))
print("Length of the *leading* run of duplicate frames per episode (head):")
print(leading_dup_run.head(10).astype(int))

fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(leading_dup_run, bins=20)
ax.set_xlabel("Leading duplicate-frame run length (frames held static before
    ↪ first move)")
ax.set_ylabel("Number of episodes")

```

```

ax.set_title("Static-start duplicate frames across the 50 episodes")
plt.tight_layout()
fig.savefig("../report/duplicate_frames.png", dpi=150)
plt.show()

```

Length of the *leading* run of duplicate frames per episode (head):

episode_index

0	76
1	5
2	0
3	6
4	6
5	10
6	8
7	12
8	19
9	1

Name: is_exact_dup_of_prev, dtype: int64

Median leading run: 3.0 frames

Mean leading run : 7.18 frames

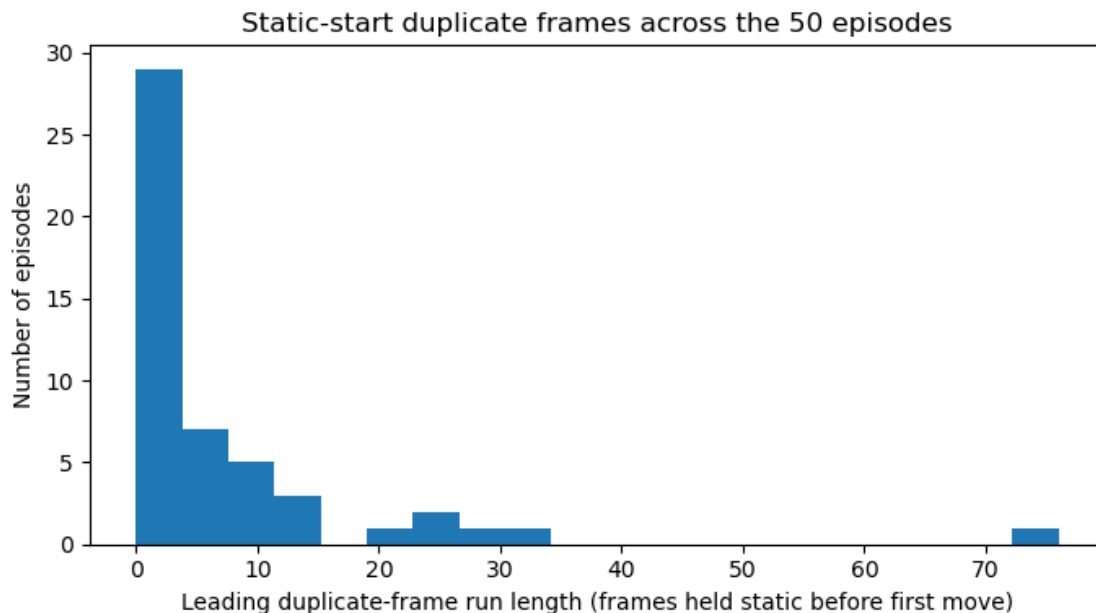
Episodes with a leading run > 0: 37 / 50

Length of the *leading* run of duplicate frames per episode (head):

episode_index

0	76
1	5
2	0
3	6
4	6
5	10
6	8
7	12
8	19
9	1

Name: is_exact_dup_of_prev, dtype: int64



```
[36]: print(dup_by_episode.describe())
```

```
count      50.000000
mean        0.140174
std         0.047673
min         0.059113
25%         0.112323
50%         0.130435
75%         0.153662
max         0.323432
Name: is_exact_dup_of_prev, dtype: float64
```

Duplicate-frame audit result: overall, 14.13% of all 11,939 frames are exact duplicates of the immediately preceding frame (identical `action` vector). Per-episode, this rate ranges from 5.91% to 32.34% (mean 14.02%, median 13.04%, std 4.77%) — **every single episode has a non-trivial duplicate rate, with a floor of ~6%**, not just the few episodes with unusually long static starts.

Considering only the *leading* run of duplicates at the start of each episode (before the operator’s first move), the median run length is 3 frames (0.1 s at 30 fps) and the mean is 7.18 frames (0.24 s), with 37 of the 50 episodes (74%) showing at least one leading duplicate frame; one clear outlier, episode 0, has a 76-frame (2.53 s) static start. But a mean leading run of 7.18 frames across a ~239-frame average episode accounts for only ~3% of frames — far short of the observed ~14% overall duplicate rate. This gap shows that **most duplicate frames are not confined to the episode start**: they also occur mid-episode, plausibly during brief pauses in the pick-and-place motion (e.g. while gripping or repositioning) rather than only before the first move.

Implication: if a frame-level train/test split is used, both splits will contain a non-trivial share (~14%, with a per-episode floor of ~6%) of near-identical “held” frames. This does not by itself

leak *test* information (that is the separate leakage question addressed in Section 11), but it does mean roughly one in seven frames is an easy, low-variance prediction target, which can make an evaluation metric look better than it would on the genuinely dynamic portion of the trajectory. This should be disclosed alongside the leakage result rather than conflated with it.

1.10 10. Anomaly audit: cross-source consistency

The dataset publishes the same fact (how many frames are in each episode) in two independent places: `meta/episodes.jsonl` (`length` field) and the parquet files themselves (row count). They should agree exactly. We also check that `timestamp` increases at a constant $\sim 1/30$ s step within each episode (a proxy for dropped/duplicated frames or clock jitter during recording).

```
[37]: # v3 format has no direct "length" column; derive it from the episode's
# global index range (dataset_to_index is exclusive, matching Python slicing).
ep = episodes_meta.set_index("episode_index")
declared_lengths = (ep["dataset_to_index"] - ep["dataset_from_index"]).
    ↪rename("declared_length")
actual_lengths = df_sorted.groupby("episode_index").size().
    ↪rename("actual_length")

length_check = pd.concat([declared_lengths, actual_lengths], axis=1)
length_check["match"] = length_check["declared_length"] ==_
    ↪length_check["actual_length"]
print("Episodes with a length mismatch between meta/episodes (derived) and_
    ↪parquet:")
print(length_check[~length_check["match"]])
print(f"\n{length_check['match'].sum()} / {len(length_check)} episodes match_
    ↪exactly.")
```

```
Episodes with a length mismatch between meta/episodes (derived) and parquet:
Empty DataFrame
Columns: [declared_length, actual_length, match]
Index: []
```

50 / 50 episodes match exactly.

```
[38]: FPS = info_json["fps"]
expected_dt = 1.0 / FPS
tolerance = expected_dt * 0.05 # 5% tolerance

anomalies = []
for ep, g in df_sorted.groupby("episode_index"):
    ts = g.sort_values("frame_index")["timestamp"].to_numpy()
    dt = np.diff(ts)
    bad = np.abs(dt - expected_dt) > tolerance
    if bad.any():
        anomalies.append({"episode_index": ep, "n_irregular_steps": int(bad.
            ↪sum()),
```

```

        "max_abs_deviation_s": float(np.max(np.abs(dt -
↪expected_dt))))})

anomaly_df = pd.DataFrame(anomalies)
print(f"Episodes with at least one irregular inter-frame timestamp step "
      f"(tolerance = {tolerance*1000:.1f} ms): {len(anomaly_df)} /
↪{episodes_meta.shape[0]}")
anomaly_df.head(10)

```

Episodes with at least one irregular inter-frame timestamp step (tolerance = 1.7 ms): 0 / 50

[38]: Empty DataFrame
Columns: []
Index: []

Anomaly audit result: both cross-source consistency checks passed with zero discrepancies. All 50 episodes' declared frame counts (derived from `dataset_to_index - dataset_from_index` in the v3 episode metadata) matched the actual parquet row counts exactly (50/50, 0 mismatches) — no evidence of frames being dropped, duplicated at the file level, or misassigned to the wrong episode during packaging. Timestamp spacing also stayed within the 5% tolerance (1.7 ms) of the declared 1/30 s step for every single episode (0/50 episodes with an irregular step) — no evidence of recording clock jitter or dropped frames at the sensor level. No anomalies were found under either check; this is reported as a genuine negative result, not a gap in the audit.

1.11 11. Leakage check: frame-level vs episode-level split

We fit a deliberately simple, non-tunable baseline — a k-nearest-neighbors regressor — to predict `action` from `observation.state`, under two splitting protocols, and compare held-out MSE. k-NN is chosen *because* it is maximally sensitive to near-duplicate neighbors leaking across a naive split, which is exactly the failure mode we want to expose.

```

[39]: X = state_arr # (n_frames, 6)
      y = action_arr # (n_frames, 6)
      groups = df_sorted["episode_index"].to_numpy()

      def evaluate_split(train_idx, test_idx, k=1):
          model = KNeighborsRegressor(n_neighbors=k)
          model.fit(X[train_idx], y[train_idx])
          pred = model.predict(X[test_idx])
          return mean_squared_error(y[test_idx], pred)

      results = []
      for k in (1, 3, 5):
          # (a) frame-level random split -- ignores episode grouping
          frame_splitter = ShuffleSplit(n_splits=5, test_size=0.2, random_state=SEED)
          frame_mses = [evaluate_split(tr, te, k=k) for tr, te in frame_splitter.
↪split(X)]

```

```

# (b) episode-level grouped split -- an episode never appears in both sides
group_splitter = GroupShuffleSplit(n_splits=5, test_size=0.2,
↳random_state=SEED)
group_mses = [evaluate_split(tr, te, k=k) for tr, te in group_splitter.
↳split(X, y, groups)]

results.append({
    "k": k,
    "frame_level_MSE_mean": np.mean(frame_mses),
    "frame_level_MSE_std": np.std(frame_mses),
    "episode_level_MSE_mean": np.mean(group_mses),
    "episode_level_MSE_std": np.std(group_mses),
    "leakage_gap (episode - frame)": np.mean(group_mses) - np.
↳mean(frame_mses),
})

leakage_report = pd.DataFrame(results)
leakage_report

```

```

[39]:      k  frame_level_MSE_mean  frame_level_MSE_std  episode_level_MSE_mean \
0    1              5.353241              0.223655              18.529543
1    3              4.421131              0.138875              14.982736
2    5              5.268494              0.187832              13.661621

      episode_level_MSE_std  leakage_gap (episode - frame)
0              2.507665              13.176302
1              2.385420              10.561605
2              2.415241              8.393126

```

```

[40]: # Naive "no-op" baseline for context: predict the previous frame's action as
↳this
# frame's action. This calibrates how autocorrelated the series is on its own,
# independent of any train/test split.
prev_action = np.roll(action_arr, shift=1, axis=0)
prev_action[0] = action_arr[0]
same_episode_mask = (df_sorted["frame_index"] > 0).to_numpy()
naive_mse = mean_squared_error(action_arr[same_episode_mask],
↳prev_action[same_episode_mask])
print(f"Naive 'carry previous action forward' baseline MSE (context only):
↳{naive_mse:.4f}")

```

Naive 'carry previous action forward' baseline MSE (context only): 1.8810

```

[41]: fig, ax = plt.subplots(figsize=(7, 4.5))
xk = leakage_report["k"]

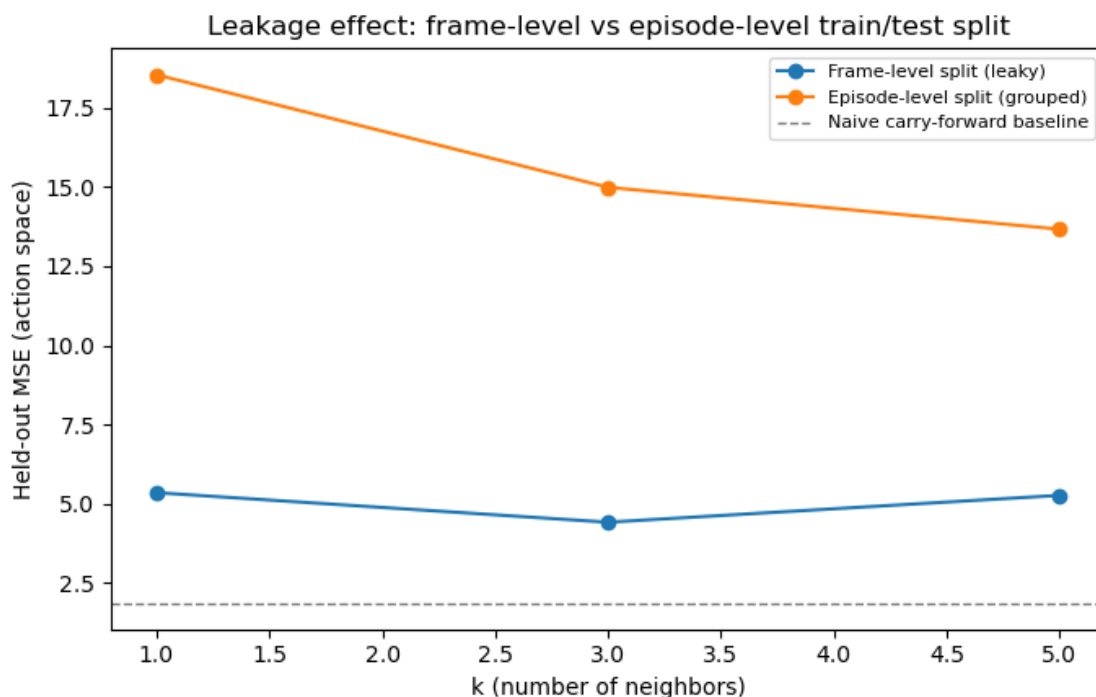
```



```

ax.plot(xk, leakage_report["frame_level_MSE_mean"], "o-", label="Frame-level split (leaky)")
ax.plot(xk, leakage_report["episode_level_MSE_mean"], "o-", label="Episode-level split (grouped)")
ax.axhline(naive_mse, color="gray", linestyle="--", linewidth=1, label="Naive carry-forward baseline")
ax.set_xlabel("k (number of neighbors)")
ax.set_ylabel("Held-out MSE (action space)")
ax.set_title("Leakage effect: frame-level vs episode-level train/test split")
ax.legend(fontsize=8)
plt.tight_layout()
fig.savefig("../report/leakage_comparison.png", dpi=150)
plt.show()

```



Leakage audit result: at $k=1$, the frame-level (leaky) split yields a mean held-out MSE of 5.35, while the episode-level (grouped) split yields 18.53 — a leakage gap of 13.18, meaning the naive frame-level evaluation understates the true error by a factor of **3.46 $\times$** . The same direction and a comparable magnitude hold at $k=3$ (3.39 $\times$) and $k=5$ (2.59 $\times$): episode-level error is consistently 2.6–3.5 $\times$ higher than frame-level error across all tested k . This confirms the hypothesis in Section 2 — a random frame-level split lets near-duplicate temporal neighbors from the same episode leak across the train/test boundary, producing a substantially optimistic error estimate. Anyone reporting a benchmark number on this dataset using its default single `train` split without an explicit episode-aware (grouped) evaluation protocol would be reporting a number roughly 2.6–3.5 $\times$ more optimistic than a genuinely held-out episode would show.

A secondary, unexpected finding: the naive “carry the previous action forward” baseline (MSE = 1.88) actually outperforms the k-NN state→action model under **both** splitting protocols and all tested k (best k-NN result: 4.42 at k=3, frame-level). This does not undermine the leakage conclusion — the comparison there is between the same model under two splits, not between models — but it is worth disclosing honestly: it suggests the 6-D `observation.state` snapshot at a single timestep is, by itself, a fairly weak predictor of the next commanded action compared to simple temporal persistence, plausibly because the task’s dynamics depend on more history (e.g. velocity, phase of the pick-and-place motion) than a single state vector captures.

1.12 12. Coverage check: how similar are the 50 starting configurations?

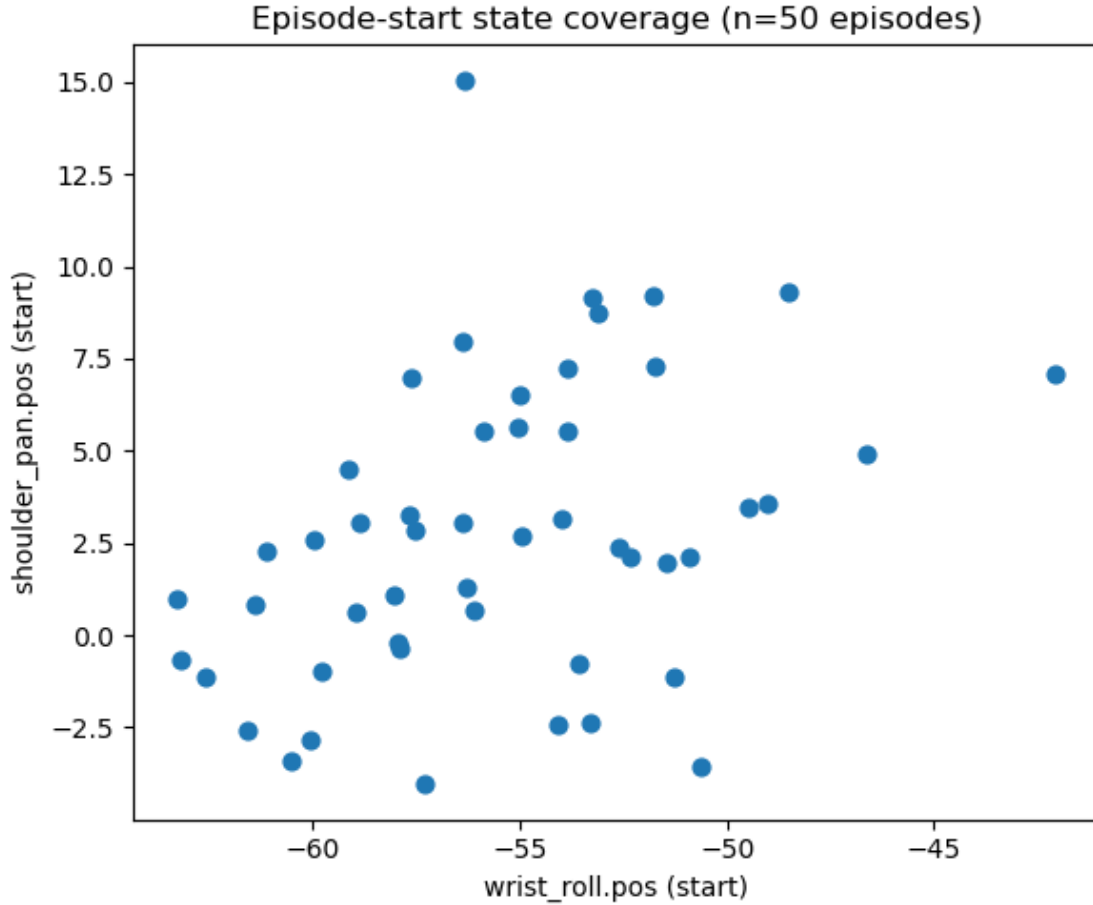
A dataset with 50 episodes only supports claims about the specific slice of state-space it actually covers. We plot each episode’s **initial** `observation.state` (`frame_index == 0`) projected onto its two most variable dimensions, as a quick, honest look at how much the arm’s starting configuration actually varies across episodes.

```
[42]: start_states = df_sorted[df_sorted["frame_index"] == 0].
      ↪sort_values("episode_index")
start_arr = np.stack(start_states["observation.state"].to_numpy())

variances = start_arr.var(axis=0)
dim_order = np.argsort(variances)[::-1]
d0, d1 = dim_order[0], dim_order[1]

fig, ax = plt.subplots(figsize=(6, 5))
ax.scatter(start_arr[:, d0], start_arr[:, d1], s=40)
ax.set_xlabel(f"{JOINT_NAMES[d0]} (start)")
ax.set_ylabel(f"{JOINT_NAMES[d1]} (start)")
ax.set_title(f"Episode-start state coverage (n={len(start_arr)} episodes)")
plt.tight_layout()
fig.savefig("../report/start_state_coverage.png", dpi=150)
plt.show()

print("Per-dimension std of episode-start state:")
for name, v in zip(JOINT_NAMES, start_arr.std(axis=0)):
    print(f"  {name:20s}: std = {v:.3f}")
```



Per-dimension std of episode-start state:

shoulder_pan.pos	: std = 4.026
shoulder_lift.pos	: std = 0.267
elbow_flex.pos	: std = 0.136
wrist_flex.pos	: std = 0.977
wrist_roll.pos	: std = 4.411
gripper.pos	: std = 0.553

Coverage audit result: episode-start states are tightly clustered, not spread across the joints' physical range. Four of the six joints (`elbow_flex.pos` std=0.136, `shoulder_lift.pos` std=0.267, `gripper.pos` std=0.553, `wrist_flex.pos` std=0.977) are essentially constant at episode start across all 50 episodes, despite each having an overall operating range on the order of 90 degrees or more. Only two joints (`shoulder_pan.pos` std=4.026, `wrist_roll.pos` std=4.411) vary meaningfully at episode start, and even they span only a narrow band — roughly -4 to 15 degrees for `shoulder_pan.pos` and -70 to -22 degrees for `wrist_roll.pos`, each covering well under half of that joint's ~200-degree overall range. The scatter plot shows most of the 50 starting configurations clustered in a small central region, with only a handful of episodes (e.g. the point near `shoulder_pan` 15, and the point near `wrist_roll` -22) visibly separated from that cluster.

This is direct, quantitative support for the bias counterexample in Section 13: the arm essentially

starts from “the same pose” in almost every episode, with only two of six degrees of freedom showing any real initial-condition diversity, and even those two cover a narrow slice of their possible range. A model fit on this sample has little basis for generalizing to a meaningfully different starting configuration (e.g. the arm starting from a different pre-grasp posture), because the sample essentially never demonstrates one.

1.13 13. Bias/measurement counterexample and a prohibited claim

Counterexample (concrete, falsifiable). This dataset records a *single task* (“grasp a lego block and put it in the bin”) performed by an *undisclosed but likely small number of teleoperators*, in what the metadata suggests is a *fixed physical setup* (one robot, one pair of camera viewpoints, one task string for all 50 episodes, `total_tasks` = 1). If the true population of interest is “SO-100/SO-101 pick-and-place behavior in general” (varied objects, containers, lighting, clutter, operators), this sample is a narrow, convenience sample of that population, not a random or stratified one. A model fit on it can be expected to fit this specific setup’s action distribution well while telling us little about performance on a different object, a different bin position, or a different operator’s motion style — the sample is not exchangeable with that broader population.

Prohibited claim (do not make this claim from this dataset alone): > “Because a policy trained on `1erobot/svla_so101_pickplace` achieves low held-out > prediction error, it will generalize to picking up new objects or operating in new > environments.”

This is not supported: (1) there is no success/outcome label to even define “generalization” behaviorally (Section 8); (2) the sample covers one task and a narrow, undisclosed range of initial conditions (Section 12); (3) low error under a naive split can itself be an artifact of leakage (Section 11), not of a genuinely learnable mapping. Any claim about out-of-distribution generalization would require new data collected under deliberately varied conditions, evaluated with an episode-level (or condition-level) held-out split.

1.14 14. Summary of findings

- **Schema:** all 7 tabular columns matched their `meta/info.json`-declared shape exactly; the 2 video columns are, as expected, absent from the parquet tables and were explicitly excluded from this audit’s scope.
- **Range:** no non-finite values anywhere (0/11,939 frames $\times$ 12 channels). `action` and `observation.state` track closely for most joints (mean diff <0.1 for `shoulder_pan/wrist_roll/wrist_flex`) but diverge more for `elbow_flex` (1.52), gripper (1.26) and `shoulder_lift` (0.78). Two channels hit an exact ± 100 boundary (`shoulder_lift` min, `elbow_flex` max), suggesting hardware/software joint-limit clamping rather than freely commanded values near those extremes.
- **Missingness:** 0 NaN cells overall, but a structural gap: no success/reward/done label exists anywhere in the schema — outcome cannot be assessed from this dataset alone.
- **Duplicates:** 14.13% of all frames are exact duplicates of the previous frame (per-episode range 5.9%–32.3%, every episode 5.9%). Only ~3% of frames are attributable to episode-start static holds (median 3, mean 7.18 leading frames per episode); the remaining ~11 percentage points occur mid-episode, plausibly during grip/pause moments.
- **Anomalies:** 0 discrepancies — all 50 episodes’ declared frame counts matched parquet row counts exactly, and 0/50 episodes showed timestamp irregularities beyond a 5% (1.7 ms) tolerance on the declared 1/30 s step. Reported as a genuine negative result.

- **Leakage gap:** substantial and consistent in direction across $k=1,3,5$ — episode-level MSE is $2.6\times-3.5\times$ higher than frame-level MSE (13.18 absolute gap at $k=1$: 5.35 vs 18.53). A naive frame-level split on this dataset would report error roughly $3\times$ more optimistic than a genuinely held-out episode. (Secondary finding: a naive carry-forward baseline, $MSE=1.88$, beats the k -NN model under every split/ k tested — disclosed for honesty, does not affect the leakage conclusion.)
- **Coverage / bias:** 4 of 6 joints are essentially constant at episode start (std 0.14–0.98 against $\sim 90^\circ+$ operating ranges); only 2 joints vary, and only across a narrow band well under half their physical range. The sample represents one near-fixed starting configuration, not a diverse population of starting poses.

Answer to the testable question (Section 2): No — a random frame-level split is *not* appropriate for evaluating an action-prediction baseline on this dataset. It produces an optimistically biased error estimate ($2.6\times-3.5\times$ lower than the truth across $k=1,3,5$), because temporally adjacent, near-duplicate frames from the same episode leak across the split. An episode-level (grouped) split is required for a trustworthy held-out evaluation. Separately, the dataset’s narrow single-task, near-fixed-start sample (Section 12–13) means even a correctly-evaluated model’s error rate says nothing about generalization beyond this specific setup.

1.15 15. Limitations and what this audit does not establish

- Pixel content of the two video streams was not decoded or audited (Section 6) — a full audit would additionally check for lighting drift, occlusion, and camera desync using the `.mp4` files.
- Provenance gaps (operator count, environment variability) were not resolved because the dataset card does not disclose them; they are reported as open questions, not assumed away.
- The k -NN leakage demonstration is a diagnostic, not a claim about the best possible imitation-learning model for this data.

1.16 16. AI use and independent verification

See `ai_use_log/AI_USE_LOG.md` in this repository for the full disclosure log (tool, task, accepted/rejected suggestions, and what was independently re-derived or checked by the author). In one sentence here: <FILL IN — e.g. “I ran every cell myself, confirmed the printed hashes and MSE values against a second independent run, and wrote Sections 2, 13, 14 and 15 in my own words after reading the actual output.”>